

# **The New Meta**

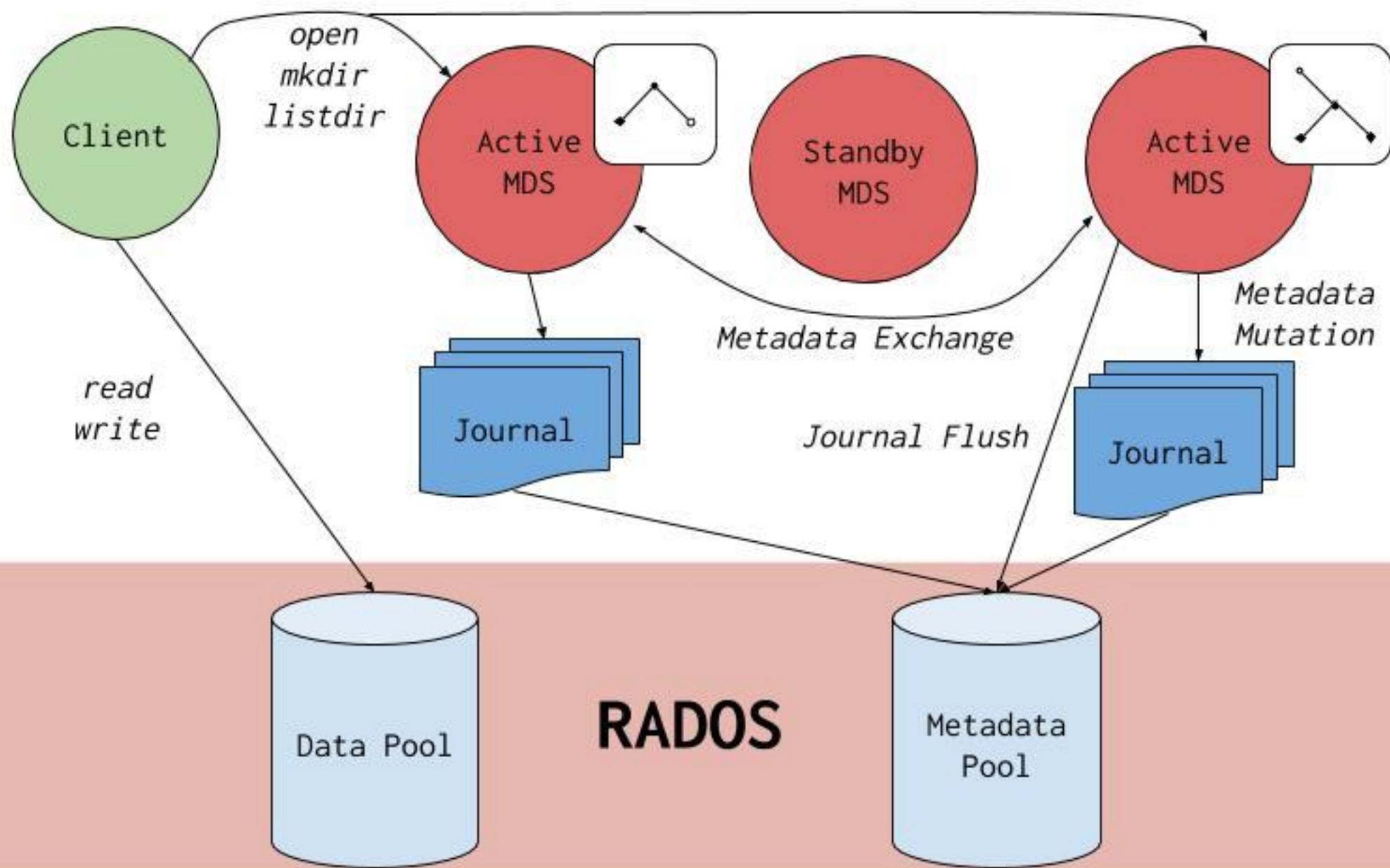
**Toward Better Metadata Handling for CephFS  
Small-File Workloads**

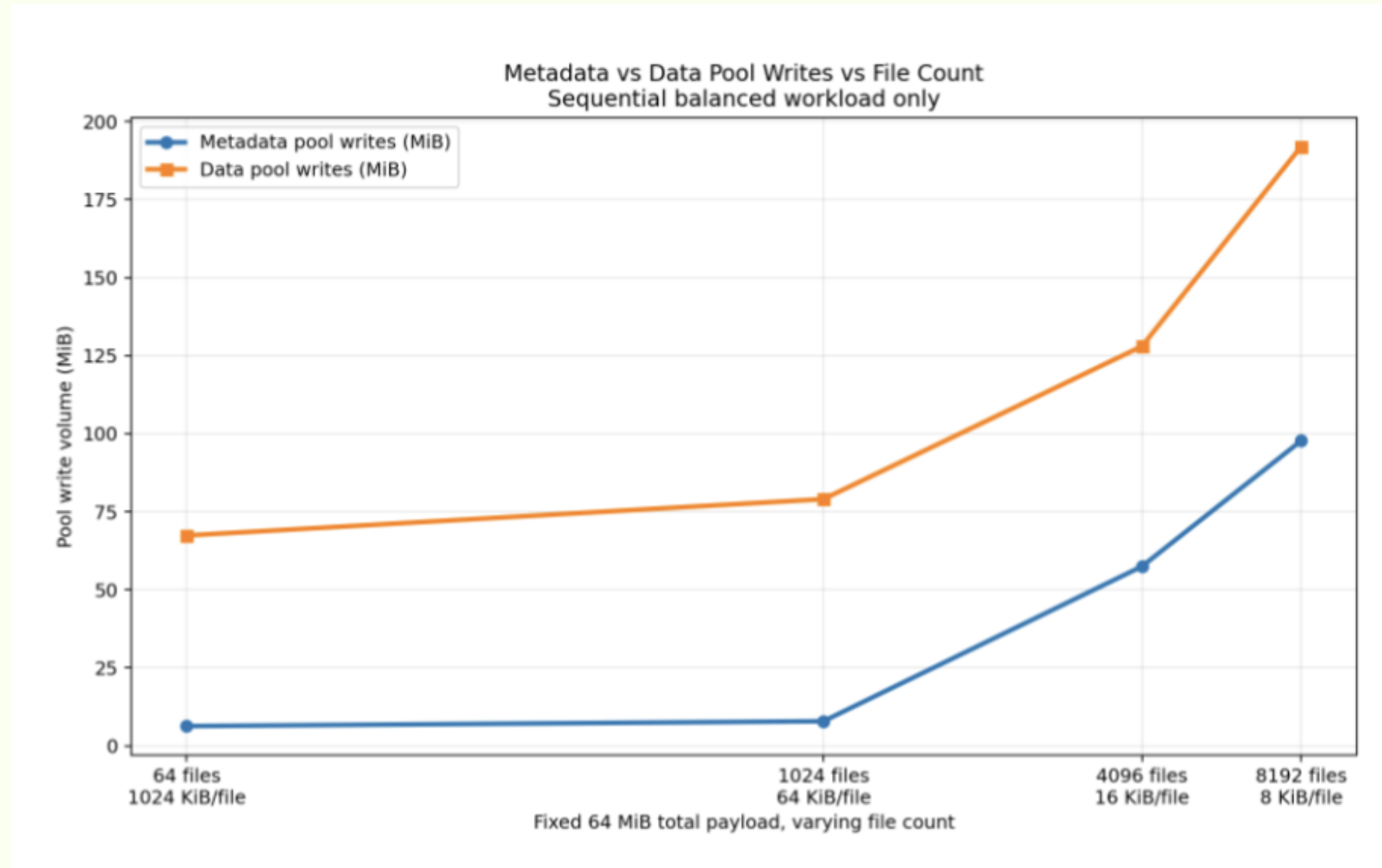
# Background: CephFS

CephFS separates data file data (Rados w OSDs) and filesystem metadata (MDS Daemons)

Metadata includes directory lookup, inode creation, stat, and subtree ownership etc. All of these will hit the MDS.

Workloads like tiny file creation and mass stats are limited by MDS operations as well as data pool.





For large number of files, we are talking about around  $\frac{1}{3}^{\text{rd}}$  of the writes

# Metadata Scaling

**Active MDS Ranks:** Multiple daemons divide directory subtrees for parallel processing.

**Dynamic Rebalancing:** CephFS migrates subtrees reactively based on load. [Reactive]

**Directory Pinning:** Users can manually export-pin directories to specific ranks to improve locality. [Risky]

**Can We Do Better?**

**Predictive Pinning:** Sliding-window hot-directory tracker

**Prepin Hotset:** Pins a small declared hot set before measured operations.

**Append Segment:** Small-file packing layer.

# Setup

node0: client-only benchmark driver.  
node1: monitor, manager, OSD, standby MDS.  
node2, node3: OSD, active MDS rank 0.

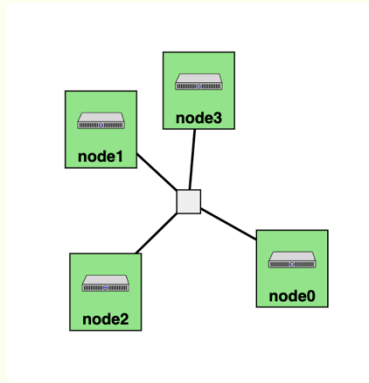
## Benchmarks:

**mdtest\_tree:** Creates directory trees to measure namespace pressure.

**sprite\_lfs\_smallfile:** Many small files across many directories for metadata stress.

**hotdirs\_zipf:** Skewed file creation toward a specific "hot" directory to test placement logic.

**varmail\_like:** Mixed create/stat/delete workload.



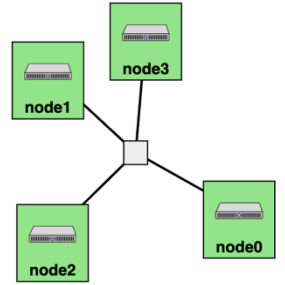


# Setup

## Baseline Policies:

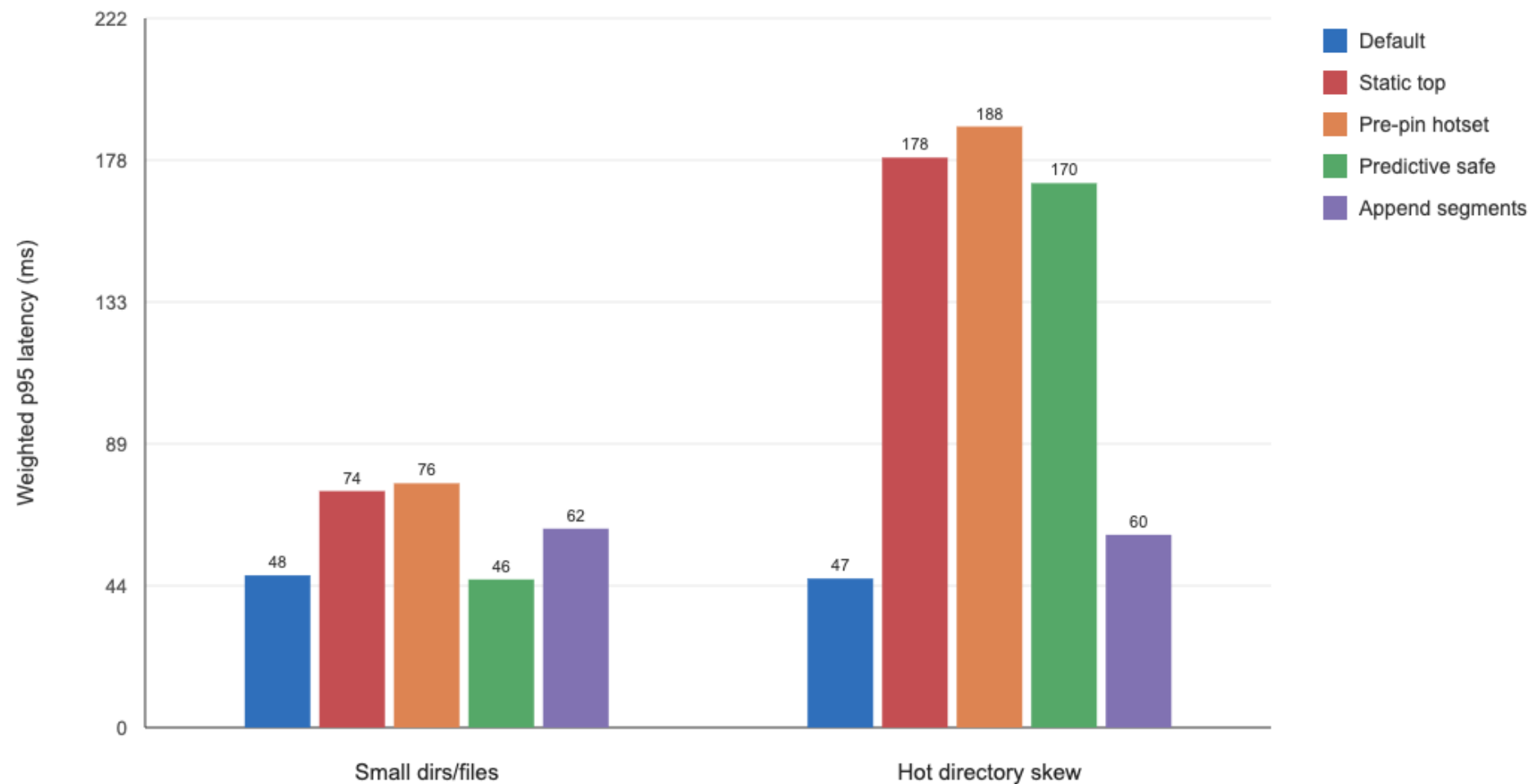
**Default**

**Static:** Aggressive pinning, just pin everything



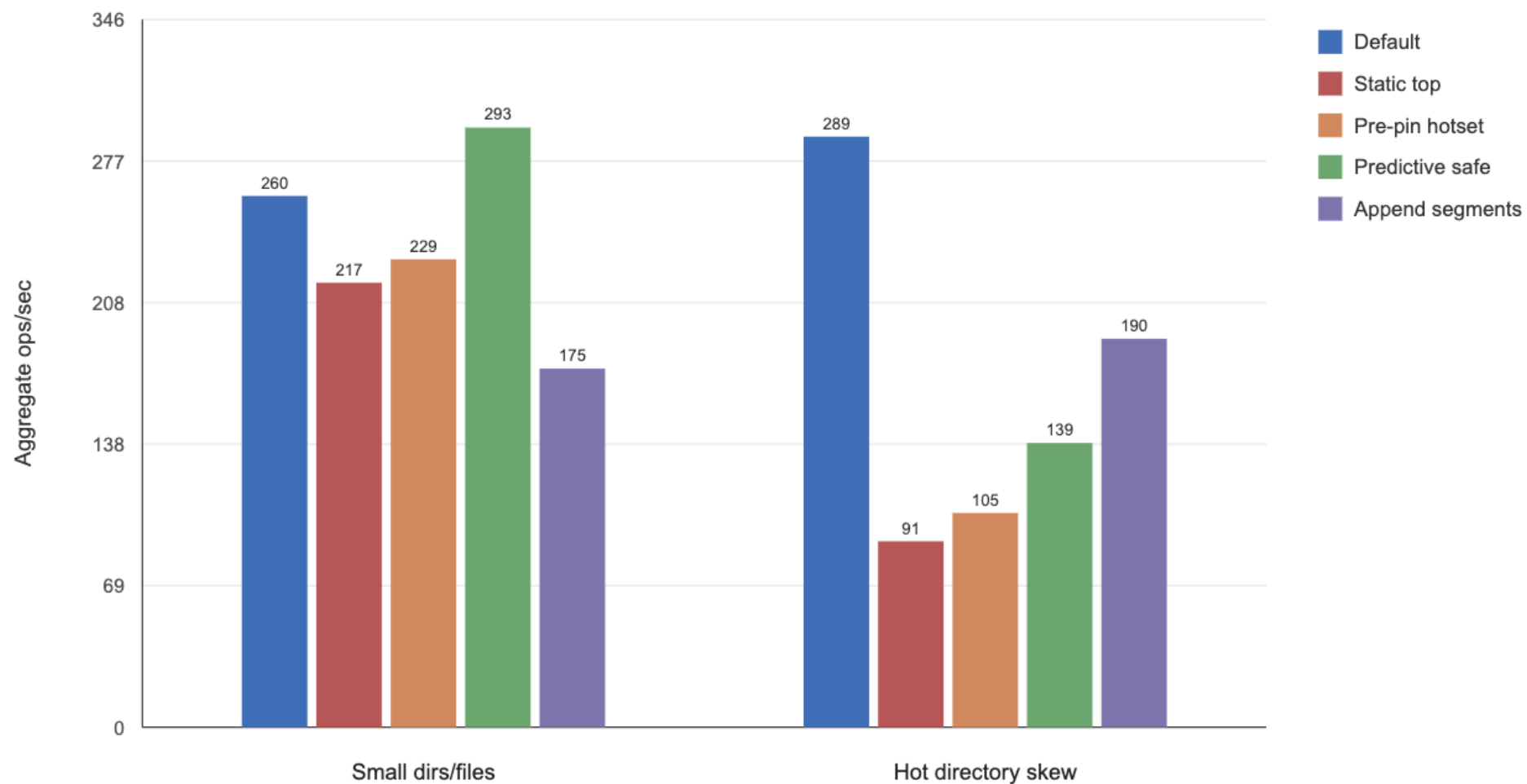
## Tail latency against default and static baselines

Lower is better. Pinning variants mainly hurt create-heavy paths.



## Throughput against default and static baselines

Retained-data 5,000-file runs, 8 workers. Static baseline here is static\_top.



# Results

| Variant         | Ops/s | Speedup | p95 Latency | Data Objects |
|-----------------|-------|---------|-------------|--------------|
| Default         | 259.7 | 1.00x   | 47.6 ms     | 5000         |
| Predictive Safe | 293.2 | 1.13x   | 46.4 ms     | 5000         |
| Append Segments | 175.3 | 0.68x   | 62.2 ms     | <b>3</b>     |
| Static Top      | 217.3 | 0.84x   | 74.0 ms     | 5000         |

# Results

Predictive pinning marginally outperforms baseline

- Pins very few times, usually pinning is BAD
- static performed worst overall

Append Segment is improving with every iteration:

Smallfile: 0.68x-0.73x default throughput.

Hotdirs: 0.66x-0.70x default throughput.

Any form of movement post write is TERRIBLE.

# The Next Week

Batching index writes and using a cheaper index format, for Append Objects.

Explore packing only cold/bulk leaving hot files as is/pinned,

Test on bigger workloads and more nodes. Try randomized workload for more defensible comparisons.

**Thank You**